



FIREBRAND

LYCEUM EDUCATION GROUP

Testing, Debugging and Quality Assurance

C# Pathway | Session 7

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Lesson Aim and Objectives

Lesson Aim: Testing, Debugging and Quality Assurance

Lesson Objectives

Apply

- Apply structured testing techniques to validate program behaviour

Analyse

- Analyse errors and debug code systematically

Evaluate

- Create test scenarios to verify functionality

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

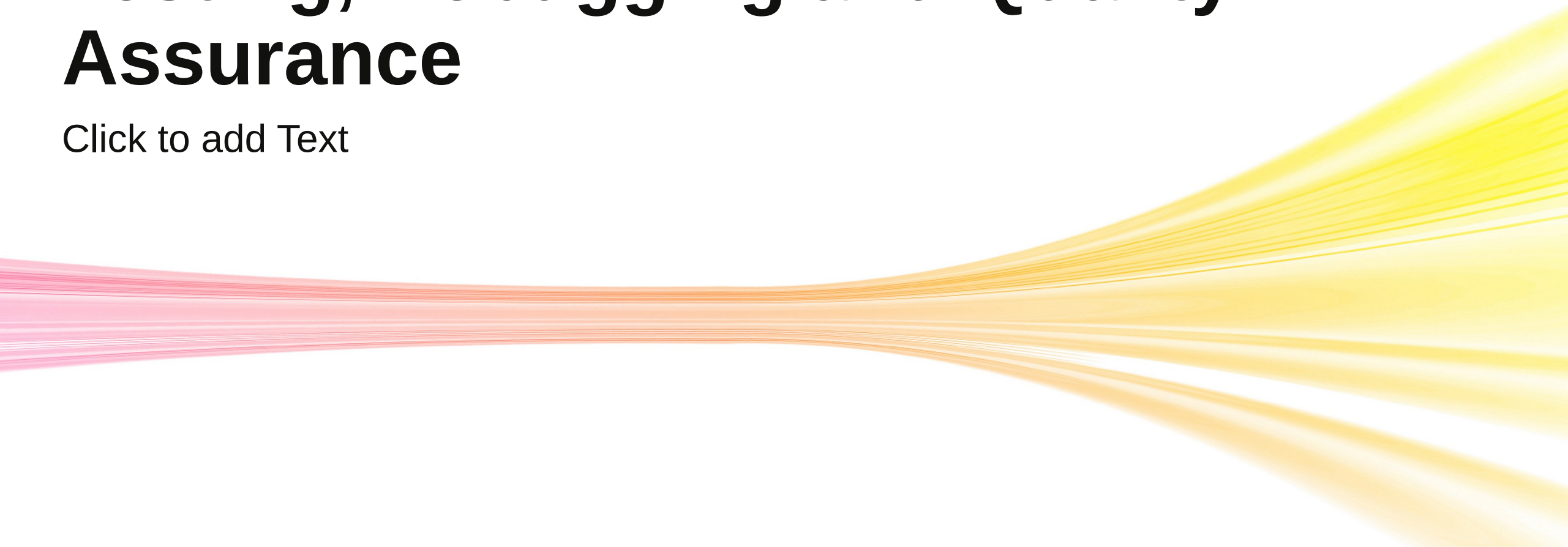
S1, **S4, S6, S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Testing, Debugging and Quality Assurance

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Testing in the SDLC

Core Purpose of Testing:

- Verifies software functionality reliably
- Ensures structural quality standards
- Prevents costly deployment failures early
- Validates user requirement alignment systematically

The SDLC Shift-Left Principle:

- Catching bugs early reduces development costs
- Fixing post-release errors is highly expensive
- Continuous validation loops stabilise core infrastructure

Introduction to Unit Testing

What is a
Unit Test?

Role in
Verification

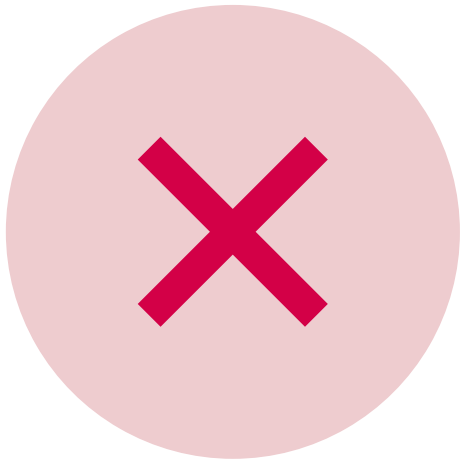
What is a Unit Test?

- Tests isolated functional code pieces
- Targets small individual components specifically
- Isolates functions from external dependencies

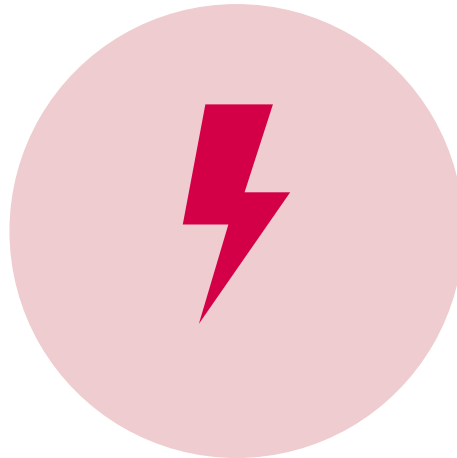
Role in Verification

- Confirms input-output accuracy predictably
- Guarantees small modules behave correctly
- Acts as automated safety nets
- Simplifies functional regression tracking instantly

The Three Common Error Types



Syntax Errors



Runtime Errors



Logic Errors

Syntax Errors

- Violates language grammatical rules completely
- Triggers immediate compilation failures

Example: Missing closing parentheses)

Runtime Errors

- Crashes during active execution phases
- Occurs despite perfect syntax layouts

Example: Dividing an integer by zero.

Logic Errors

- Runs smoothly without crashing once
- Produces incorrect mathematical outputs

Example: Adding values instead of multiplying

Early Issue Detection

The Cost of Delay:

- Bugs compound over time rapidly.
- Hidden logic flaws corrupt system data.
- Early fixes protect project delivery timelines

Automated Validation Results:

- Improves code base stability permanently.
- Boosts development team confidence before deployment.
- Ensures reliable continuous integration pipelines

Spot the Logic Error

Flawed Script:

```
public static double Average(int[]  
numbers) {  
    int total = numbers.Sum();  
    return (double) total / numbers.Length  
- 1; } // bug!
```

Corrected Script:

```
public static double Average(int[]  
numbers) {  
    int total = numbers.Sum();  
    return (double) total /  
numbers.Length; }
```

This code runs without crashing — why is it still wrong?

The code is wrong because it **always outputs a result that is exactly 1 lower than the true mathematical average**

Writing a Simple Unit Test

- A unit test checks that one function behaves correctly for a given input
- C# provides `Debug.Assert()`, which flags an error if a condition is false

Code Example:

```
public static List<string> AddItemToCart(List<string> cartList, string itemName, int
maxCapacity) {
    /**
     * Adds a unique item to a shopping cart list up to a maximum threshold size.
     */
    if (cartList.Count >= maxCapacity) {
        throw new InvalidOperationException("Cart is full!"); }
    if (cartList.Contains(itemName)) {
        Console.WriteLine(itemName + " already in cart.");
        return cartList; }

    cartList.Add(itemName);
    return cartList; }
```

Structured Debugging Lab



ACTIVITY



Objective: Diagnose and fix a program containing multiple error types

Tasks / Requirements:

- Run the provided script(s) and record the first error message shown
- Fix the error, re-run, and repeat until the script executes correctly
- For each fix, classify the error as **syntax**, **runtime**, or **logic**

Key Point: Evidence a structured approach: identify, correct, re-test, and classify

Extension Activity: Write two additional unit tests covering edge cases for the fixed function



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Test Design Workshop



What is a Test Case?



Designing Test Cases

Normal Case

Edge Case

Invalid Case

Normal Cases (The Golden Path)

- Standard expected input values
- Valid everyday operational data
- Follows the primary path
- Verifies core algorithmic accuracy
- Confirms base execution functionality

Edge Cases (The Boundary Tipping Points)

- Extreme system limit thresholds
- Minimum possible valid boundaries
- Maximum allowable size metrics
- Empty collection states ([])
- Numerical identity parameters (0)
- Catches off-by-one logical flaws

Invalid Cases (The Safety Shield)

- Erroneous input data formats
- Incorrect variable data types
- Negative numbers in calculations
- Values exceeding capacity constraints
- Triggers graceful validation handlers
- Prevents complete runtime crashes

Designing Test Scenarios for Robust Quality Assurance

The Mission:

- **Create a Test Suite:** Plan and design comprehensive test coverage for an existing program
- **Map Input States:** Structure test assertions across varied operational conditions
- **Define Expectations:** Predict exact numerical outputs or errors before running the code

Expected Outcome:

- A **well-defined, structured test matrix** covering normal use cases, extreme boundaries, and invalid error inputs

Practical Test Case Application

Here is how these three categories apply to a function checking movie age restrictions (18+ gate, maximum age limit 120):

Category	Input Value	Expected Program Action
Normal Case	25	Standard processing allowed
Edge Case	18	Threshold boundaries pass successfully
Edge Case	0	Catches lowest boundary value
Invalid Case	-15	Rejects negative numeric value
Invalid Case	"eighteen"	Rejects text data type

Validation Activity



ACTIVITY



Objective: Execute test cases and record the results

Tasks / Requirements:

- Run the provided script and enter the values in your test cases, covering:
 - Normal
 - Edge Cases (Upper & Lower)
 - Invalid
- Compare 'Actual Results' with 'Expected Results'
- Document your findings clearly

Key Point: Demonstrate accurate execution and validation of program behaviour



ACTIVITY



Documenting Your Evidence

Test ID	Category	Sample Input	Expected Outcome	Actual Result	Status (Pass / Fail)
TC-01					
TC-02					
TC-03					



FIREBRAND

LYCEUM EDUCATION GROUP

Quality Discussion

The Strategic Impact of Software Testing

System Reliability

- Prevents unexpected operational failures
- Isolates boundary edge cases early
- Eliminates critical system crash vectors
- Secures stable, continuous background deployment

Expected Outcome

- **Articulate value:** Explain precisely why rigorous validation frameworks distinguish professional software architectures from fragile scripts

User Trust and Experience

The Direct Business Cost of Broken Code:

Building User Confidence:

- **Flawless user interactions:** Smooth interfaces create polished product experiences.
- **Data protection:** Strict verification gates prevent state errors from corrupting personal database fields.
- **Brand reputation safety:** Unhandled runtime crashes drive customers to competitor products immediately.

The Reality of Churn:

- Users rarely blame their hardware; they uninstall buggy software permanently

Maintainability and Future Growth

Regression Protection:

- **Automated safety anchors:** Code coverage suites protect existing legacy systems while adding new updates
- **Fearless refactoring:** Developers safely restructure old code engines without breaking core behaviour blocks
- **Low system coupling:** Clear functional division minimises complex code dependencies

Long-Term Savings:

- Catching a logic flaw during testing takes minutes; finding it post-release costs business infrastructure pipelines millions



Stretch and Challenge

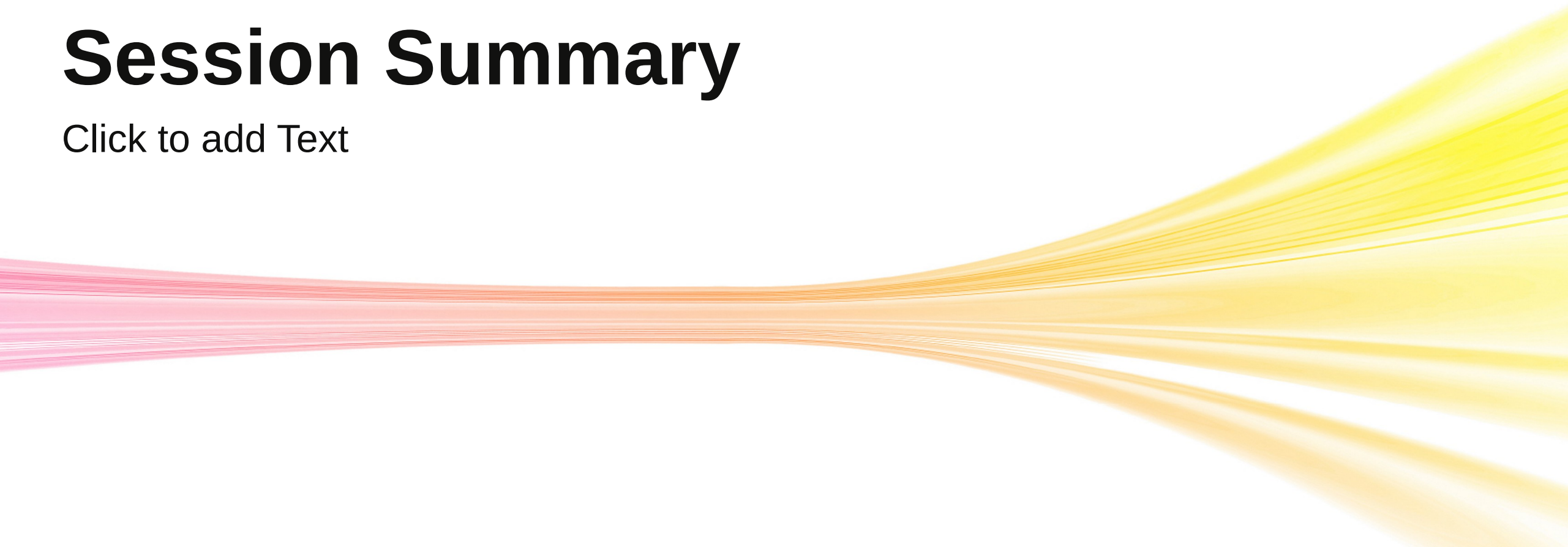
The Mission:

Identify Critical Gaps in Validation Suites

- **Task 1:** Review an incomplete test suite
- **Task 2:** Identify gaps in coverage
- **Task 3:** Propose additional test cases to improve completeness

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Testing, Debugging and Quality Assurance

Lesson Objectives

Apply

- Apply structured testing techniques to validate program behaviour

Analyse

- Analyse errors and debug code systematically

Evaluate

- Create test scenarios to verify functionality

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, **S4, S6, S7**, S10,
S11, S12, S16

Behaviours

B2, B3

